

T5 Evidence Check: Observability & Governance

Each claim in the two editions, traced to the experiment and the recorded evidence behind it, with a status: supported, qualified or not tested.

T5

T5 · Production AI Engineering

4 min read · Published 2026-09-30 · Evidence Check

01 · The question we tested

This is a **forensic-reconstruction and failure-injection POC**. After an autonomous production action, **can the platform reconstruct and prove the complete causal chain from intent to physical side effect, and how much of it can application logs and distributed traces reconstruct on their own?** The POC runs one production rollback in 15 scenarios across 12 experiments (one of them the normal happy path), with a concurrent second execution, and observes every run three ways: application logs (L0), logs plus OpenTelemetry traces (L1) and governed execution lineage (L2). Each layer answers thirteen questions, scored against ground truth from the systems of record.

02 · What ran

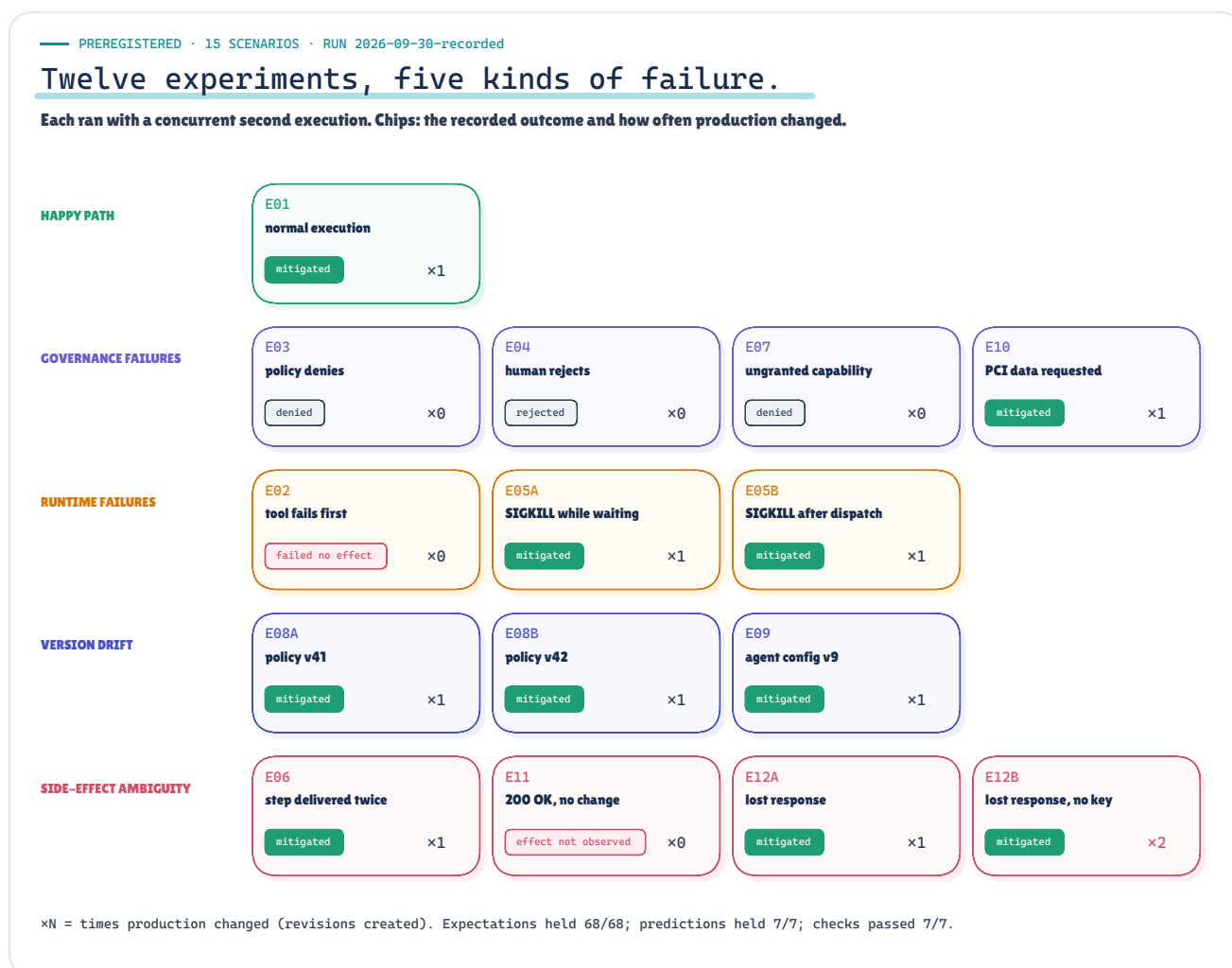


Figure 1. The recorded run: fifteen scenarios in five groups, with each one's outcome and production-change count.

Measured: every scenario's recorded outcome and production changes · run 2026-09-30-recorded

15 scenarios covering 12 experiments; 62 processes, 2 real SIGKILLs, 32 HTTP requests to the deployment API, 30 model calls (qwen3:8b), 399 evidence events, 573 spans, 605 log lines. The drift probe added 20 calls. The split between real and simulated is in Real vs simulated. Every observed value is in the Run report. Every scenario, through each layer, with its input, output, answers against truth, recorded steps and data lineage, is in the Lab Console.

03 · Integrity of the evidence

Check	Result
preregistration frozen before the run (digest in <code>manifest.json</code>)	<code>sha256:08cc152fc873...</code>
run checks (<code>checks.json</code>)	7 of 7 passed
per-scenario expectations	68 of 68 held (missed: none)
reconstruction predictions	7 of 7 held
evidence chains and anchors verified	15 of 15 scenarios
replay with Ollama unreachable (<code>2026-09-30-recorded-replay</code>)	60 answers from tape, 0 misses, identical: yes
drift probe replay identical	yes
POC test suite, model unreachable	31 of 31 passed

04 · Claims

Status: **Supported**, the recorded evidence shows it; **Qualified**, supported with a stated bound; **Not tested**, argued, not measured.

#	Claim	Experiment	Evidence	Status
C1	Application logs answered most "what happened" questions correctly.	all	L0 149/195; Q5 15/15, Q9 15/15, Q11 15/15 · <code>reports/comparison.json</code>	Supported
C2	Logs did not record on whose behalf the agent acted, or which configuration produced the proposal.	all	L0 Q2 0/15, Q4 0/15	Supported
C3	Adding standard traces answered no additional question; it removed timestamp joins.	all	L1 149/195; timestamp joins 15 → 0	Supported
C4	The execution lineage answered all thirteen questions in every scenario.	all	L2 195/195	Qualified: a coverage test, not a benchmark; its schema was designed around these questions
C5	A tool's success response was recorded by the logs as mitigation while production was unchanged; the read-back caught it.	E11	"mitigated?" L0 answered yes (wrong), L2 answered no (correct); revision 184 → 184 · <code>scenarios/e11-false-success/</code>	Supported
C6	With an idempotency key, a lost response and a retry changed production once.	E12a	2 requests, 1 change, revision 184 → 185, 1 replay	Supported
C7	Without the key, the same retry changed production twice with the same final version.	E12b	2 changes, revision 184 → 186, version v4.17.2	Supported
C8	A runtime killed after dispatch resolved the unknown outcome by lookup, not by resending.	E5b	1 reconciliation, 1 attempt, 1 change	Supported
C9	A step delivered twice did not change production twice.	E6	2 requests, 1 change	Supported

#	Claim	Experiment	Evidence	Status
C10	The evidence records which policy version governed, with its digest and obligations.	E8a/E8b	v41 quorum 1 · v42 quorum 2	Supported
C11	The evidence records which agent configuration and prompt template produced a proposal.	E1/E9	incident-agent-prod@8 / incident-agent-prod@9; incident-remediation@17 / incident-remediation@18	Supported
C12	A request for restricted data was denied and none of it reached the model or any record.	E10	denied payments.customer_transactions; canary hits 0 in 225 files	Supported
C13	An ungranted capability was refused by policy and at the gateway, with nothing sent.	E7	1 gateway denial, 0 attempts	Supported
C14	Telemetry lost data when processes were killed; evidence did not.	E5a/E5b	13 orphaned spans, 2 metric dumps lost; chains intact	Supported
C15	Head sampling would have removed most executions' traces.	all	kept at 10%: 1/15; at 1%: 0	Supported (computed on recorded trace ids)
C16	The hash chain plus external anchor detected edits, rewritten chains and truncation; it cannot detect an attacker who also rewrites the anchor.	tamper	detected 3 of 3; T4 detected: no · reports/tamper.json	Qualified: needs an anchor under separate control
C17	No credential appeared in any log, span, tape or evidence event.	all	credential hits 0	Supported
C18	A configuration change moved the action mix while each proposal stayed within policy.	D1	rollbacks 7 → 0; 8/10 changed	Qualified: 20 calls, one seed; a probe
C19	The trace continued across a SIGKILL restart.	E5b	one trace id for the whole execution (end-to-end test test_crash_after_dispatch_is_reconciled_not_repeated)	Supported

#	Claim	Experiment	Evidence	Status
C20	Timestamp joins become ambiguous under concurrency.	none	not observed: the executions were on different services	Not tested
C21	Idempotency, not the lineage, prevented the duplicate; the lineage is what records that the two attempts were one action and one external transaction.	E12a/E12b	with key: 1 change, 1 transaction for the key; without: 2 changes, both recorded by the lineage (revision 184 → 186)	Supported
C22	The three layers observed one shared action path; the governed path added one behaviour, reading the world back before declaring success.	all	disclosed by design (<code>profile</code> tags on log lines, <code>lineage/investigate.py</code>)	Supported (method)
C23	Verification holds when an API applies a change later than it acknowledges it.	none	the next experiment ("accepted now, applied later")	Not tested

05 • What is not claimed

No claim of legal compliance; no claim that SQLite is an appropriate evidence store; no claim about real Kubernetes controllers' asynchronous behaviour; no claim that the event schema is universal; no claim about approval latency or human error; no claim that the drift probe detects drift in production.

Evidence

Claims

Observability

Governance

PRODUCTION AI ENGINEERING · THE SERIES SO FAR

FOUNDATION

F1 · MCP Tool Sprawl

Your AI agent has 500 MCP tools. Now what?

FOUNDATION

F2 · Layered Architecture

Your agent works in a demo. Why does it break in production?

FOUNDATION

F3 · Headless AI

Your AI shouldn't live inside the UI

TRUST

T1 · Agent Identity

Who is acting, and on whose authority?

TRUST

T2 · Authorization & Policy

What is this agent actually allowed to do?

TRUST

T3 · Human-in-the-Loop

Authorized. Should it still act?

PREVIOUS

AI Control Plane

Who coordinates identity, policy, approval and budgets across every agent? (in progress)

CURRENT

T5 · Observability & Governance

Your agent changed production. Can you prove why?

NEXT

Production Agent Platform

The final reference architecture (planned)

[The full learning_map_\(Start Here\)](#).

T5 · Production AI Engineering

Production AI Engineering · T5 · Evidence. Every measured number here is substituted at build time from observability_governance_poc/runs/2026-09-30-recorded/facts.json; the enterprise systems are simulated and the run is deterministic. Figures carry their provenance.

[Medium edition](#) [Technical deep dive](#) [POC README](#) [Run summary](#) [Lab Console](#) [Run report](#)
[QA report](#)